

Retrieval-Augmented Generation (RAG) - Machine Learning Knowledge Assistant

By: Athreya Kidambi

1) Objective:

The objective is to design and implement an end-to-end Retrieval-Augmented Generation (RAG) system that can accurately answer Machine Learning-related questions using only the content available in a reference textbook. The system is expected to retrieve relevant information from the source document and generate grounded, context-aware answers with minimal hallucination. This project demonstrates the practical integration of document processing, semantic search, and large language models into a unified question-answering pipeline.

The knowledge source used for this implementation is the book *Introduction to Machine Learning with Python* (PDF format), which contains structured explanations of supervised learning, model complexity, and generalization concepts.

2) Dataset Understanding and Preprocessing (Part 1):

The dataset used is a PDF version of a Machine Learning textbook, which contains structured chapters, explanatory text, and conceptual discussions relevant to classical Machine Learning algorithms. The first step of the pipeline involved loading and exploring the PDF document to understand its structure and content quality.

Due to compatibility issues encountered with standard PDF loaders in the Colab environment—specifically arising from indirect dependencies on `torchcodec` and `ffmpeg`—the document was loaded using the `pypdf` library. Text was extracted page by page, and each page was converted into a standardized LangChain Document object while preserving metadata such as page number and source file path. This ensured complete ingestion of the document while maintaining traceability between retrieved content and the original source.

Exploratory inspection showed that the document contained a total of 387 pages, confirming that the entire book was successfully loaded. Minor formatting artifacts such as line breaks and hyphenation were observed, which are common in PDF text extraction and were handled downstream during retrieval and generation.

To enable effective semantic retrieval, the extracted text was segmented into smaller overlapping chunks using a recursive character-based text splitter. A chunk size of 800

characters with an overlap of 120 characters was selected, aligning with the suggested range of 500–1000 tokens and overlap between 50–150 tokens. This approach balances contextual completeness with retrieval precision, resulting in 1,143 text chunks used for embedding and indexing.

3) Embedding and Vector Database (Part 2):

To convert textual chunks into a form suitable for semantic search, dense vector embeddings were generated using the SentenceTransformers model all-MiniLM-L6-v2. This model was selected due to its strong performance in capturing semantic similarity while remaining computationally efficient, making it well-suited for large document collections.

Each text chunk produced during preprocessing was embedded into a fixed-length numerical vector. These vectors were then stored in a FAISS (Facebook AI Similarity Search) vector database. FAISS was chosen as the vector store because of its fast similarity lookup performance and its recommendation in the guidelines. The final FAISS index contained 1,143 vectors, matching the total number of generated chunks and confirming consistency across preprocessing and indexing stages.

Similarity search was implemented using cosine similarity over vector embeddings, enabling efficient retrieval of the most relevant text chunks corresponding to an input query. This establishes the foundational retrieval capability of the RAG system.

4) Retrieval Pipeline (Part 3):

The retrieval pipeline enables query-based access to the indexed knowledge. When a user submits a question, the query is first converted into an embedding using the same SentenceTransformers model to maintain vector space consistency. The embedded query is then compared against the FAISS index to retrieve the most semantically similar text chunks.

To evaluate retrieval effectiveness, multiple values of k were experimented with, and a value of $k = 3$ was selected for answer generation. This choice balances contextual relevance and prompt length limitations while minimizing noise from unrelated content. For the query *“What is overfitting in machine learning?”*, the system successfully retrieved chunks that explicitly define overfitting, discuss generalization trade-offs, and contrast overfitting with underfitting. The retrieved content directly aligns with the textbook’s explanation, providing strong evidence of effective semantic retrieval and satisfying the requirement for example retrieved chunks.

5) Answer Generation Using RAG (Part 4):

In the answer generation stage, retrieval is integrated with a large language model to produce grounded responses. The retrieved text chunks were concatenated and injected into a carefully designed prompt that explicitly instructs the model to answer only using the provided context.

The model used for generation is FLAN-T5-Large, a sequence-to-sequence transformer well suited for instruction-following tasks. The prompt structure includes a clear separation between context and question and emphasizes concise, relevant responses based strictly on retrieved content. This design ensures minimal hallucination and enforces grounding, as required.

Upon generation, the model produced an answer explaining overfitting in terms of model complexity and generalization behaviour. The response directly reflects the retrieved textbook content, without introducing external knowledge or unsupported claims. This demonstrates successful grounding and confirms the effectiveness of the RAG approach in controlling model behaviour through retrieval-based context.

6) End-to-End RAG Application (Part 5):

All components of the system—document ingestion, text chunking, embedding generation, vector storage, semantic retrieval, and LLM-based answer generation—were integrated into a single end-to-end pipeline. The system supports real-time question answering by dynamically retrieving relevant context and generating responses on demand.

Multiple queries can be issued to the system without reprocessing the document, as embeddings and the FAISS index are reused. This satisfies the requirement for a functional, demonstrable RAG application capable of handling multiple queries.

7) Conclusion:

This project successfully demonstrates the design and implementation of a complete Retrieval-Augmented Generation system for Machine Learning knowledge assistance. By combining robust preprocessing, efficient semantic retrieval, and controlled LLM generation, the system produces accurate and grounded answers directly sourced from the reference textbook. The implementation adheres closely to the prescribed tasks, recommended tools, and evaluation criteria, and provides a practical blueprint for building reliable, document-grounded question-answering systems.